

Patrón arquitectónico: MVVM

(Model–View–ViewModel)

Patrones de diseño (POO)

Propósito

Separar responsabilidades para construir interfaces de usuario **mantenibles y testeables**:

- **View**: muestra estado y captura eventos de usuario.
- **ViewModel**: transforma datos en *estado de vista* y orquesta acciones.
- **Model**: reglas de negocio y datos (dominio, acceso a datos, servicios).

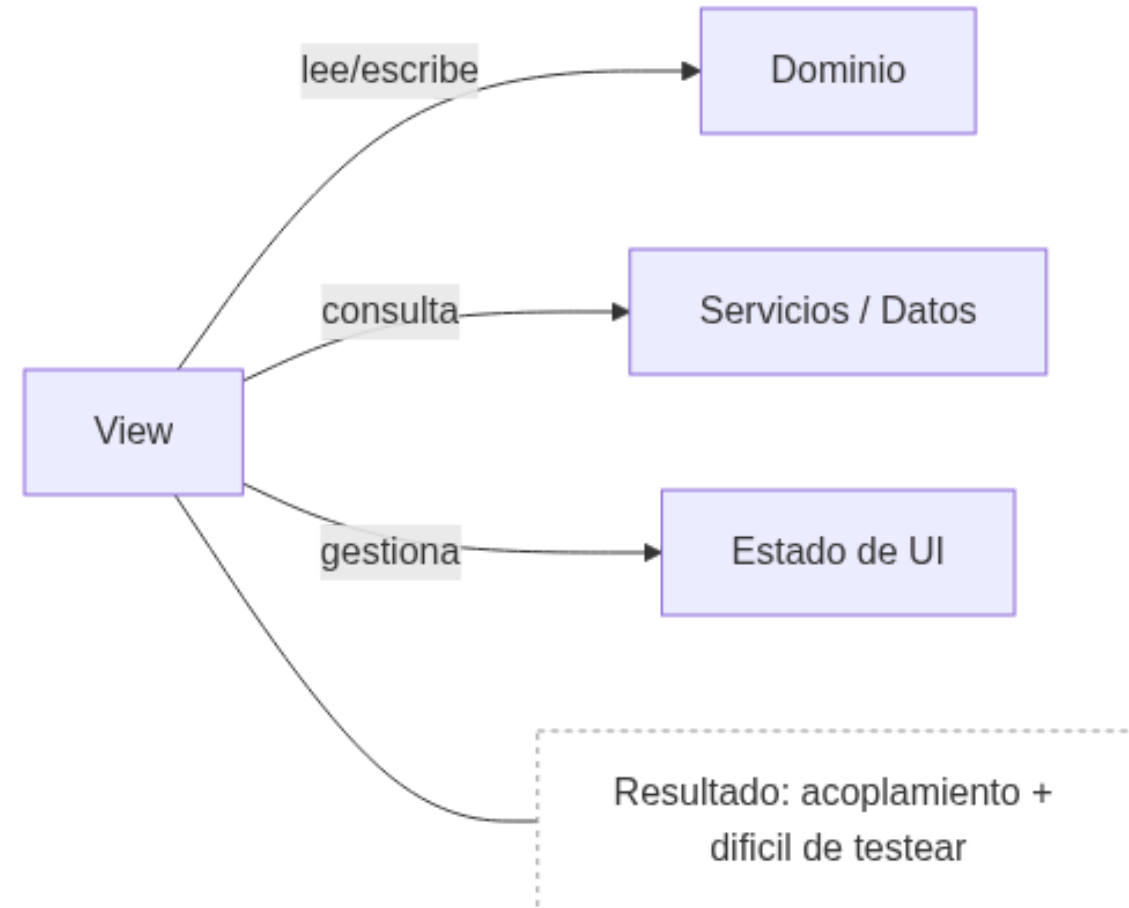
Objetivo: que la UI cambie sin romper la lógica, y que la lógica se pruebe sin UI.

Problema

Cuando la **Vista** acumula lógica y estado:

- Aumenta el **acoplamiento** entre UI y dominio.
- Los cambios de UI obligan a tocar lógica.
- Las pruebas se vuelven caras (necesitas simular UI).
- El estado es frágil: depende del ciclo de vida/pantalla/instancia.

Resultado típico: una “vista dios” difícil de evolucionar.

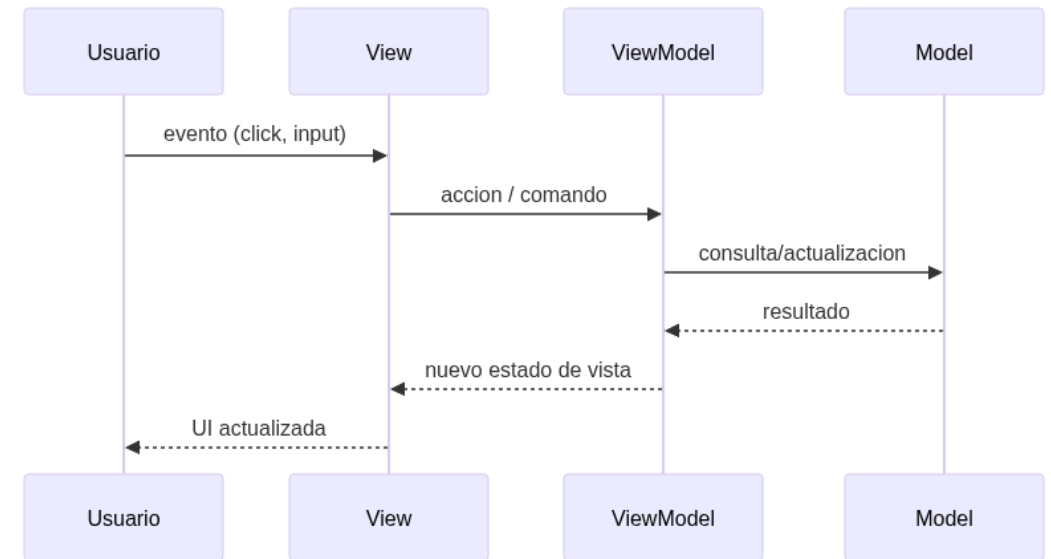


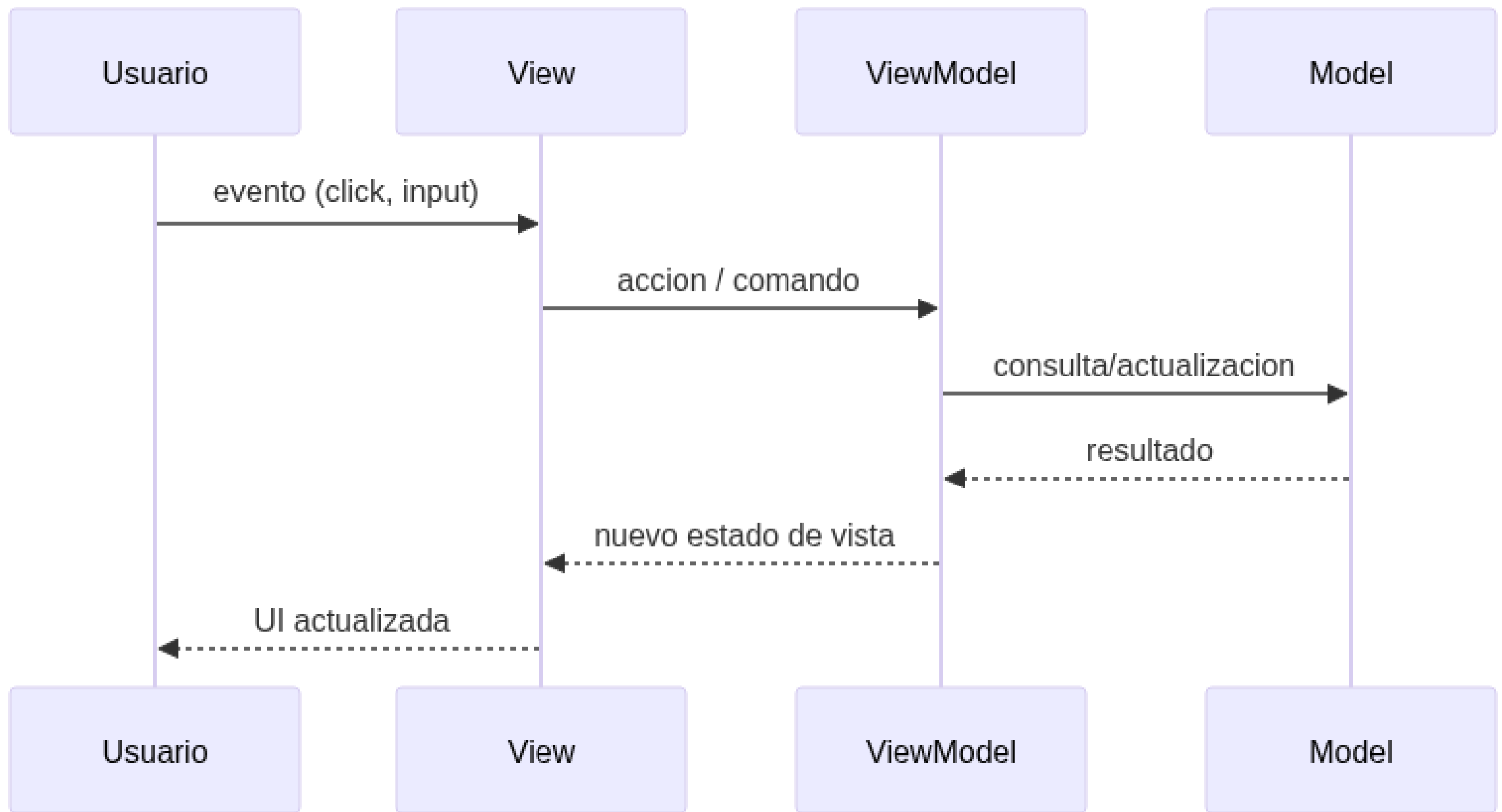
Solución

Mover fuera de la Vista lo que no es “pintar” ni “capturar eventos”:

- La **ViewModel** mantiene el **estado observable** que la View representa.
- La View envía **eventos/acciones** a la ViewModel (comandos).
- La ViewModel habla con el **Model** (casos de uso, repositorios, servicios).

Regla práctica: *la View no decide; solo delega y representa.*





Analogía en el mundo real

Restaurante:

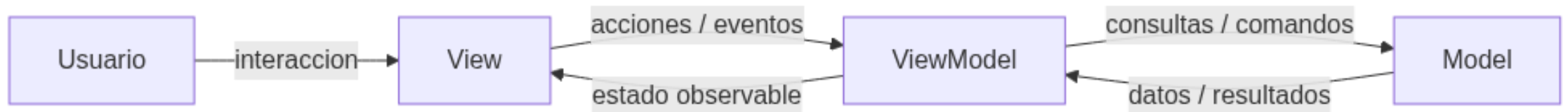
- **View** = camarero: toma pedidos y entrega platos (interacción).
- **ViewModel** = jefe de sala: traduce pedidos a órdenes claras, coordina.
- **Model** = cocina y almacén: recetas, preparación, inventario (reglas y datos).

El camarero no cocina; el jefe de sala no sirve mesas.

Estructura

Roles y flujo de comunicación (modelo mental):

- View $\rightleftarrows$ ViewModel (eventos $\leftrightarrow$ estado)
- ViewModel $\rightleftarrows$ Model (consultas/actualizaciones)



Aplicabilidad

Úsalo cuando:

- Hay UI con **estado** que cambia frecuentemente.
- Quieres **pruebas unitarias** de la lógica de presentación.
- Necesitas separar equipos/roles (UI vs lógica/datos).
- Esperas cambios de UI (rediseños, nuevas vistas) sin reescribir dominio.

Evítalo si la UI es trivial (sobrearquitectura).

Pros y contras

Pros

- Mejor **separación de responsabilidades**.
- ViewModel más **testable** que una vista con lógica.
- Menos acoplamiento: la UI evoluciona con menos impacto.

Contras

- Más piezas (ViewModel/estado/observables): mayor complejidad inicial.
- Riesgo de ViewModel "gigante" si se mete lógica de dominio ahí.
- Si no se define bien el flujo de datos, puede haber duplicación de estado.